Atividade Somativa 1 - Técnicas de Machine Learning

Introdução

O objetivo deste trabalho é utilizar técnicas de Machine Learning para prever a variável WinShares presente no dataset NBA Players Stats.

A variável WinShares representa uma estimativa da contribuição de um jogador para as vitórias de sua equipe ao longo de uma temporada.

Como a variável alvo é numérica e contínua, este problema é classificado como um problema de regressão supervisionada.

```
#importando planilha csv para jupyter notebook
#biblioteca pandas ótima para lidar com planilhas
#salva a planilha em df

import pandas as pd

df = pd.read_csv('nba_stats.csv')

print(df.shape)

df.head()
```

```
(24624, 22)
```

	Ano	Posicao	Idade	Jogos	TrueShootingPercentage	FreeThrowRate	OffensiveWinShares	DefensiveWinShares	WinShares	FieldGoa
0	1950	G-F	31.0	63	0.368	0.467	-0.1	3.6	3.5	1
1	1950	SG	29.0	49	0.435	0.387	1.6	0.6	2.2	1
2	1950	SF	25.0	67	0.394	0.259	0.9	2.8	3.6	1
3	1950	F	24.0	15	0.312	0.395	-0.5	-0.1	-0.6	
4	1950	F	24.0	13	0.308	0.378	-0.5	-0.1	-0.6	

5 rows × 22 columns

Análise Inicial dos Dados

Nesta etapa foi realizado o carregamento do dataset e uma análise preliminar da estrutura dos dados, incluindo quantidade de registros, tipos de variáveis e identificação de valores ausentes.

```
#informações gerais da planilha
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24624 entries, 0 to 24623
Data columns (total 22 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Ano                             24624 non-null  int64
 1   Posicao                         24624 non-null  object
 2   Idade                           24616 non-null  float64
 3   Jogos                           24624 non-null  int64
 4   TrueShootingPercentage          24538 non-null  float64
 5   FreeThrowRate                   24525 non-null  float64
 6   OffensiveWinShares              24585 non-null  float64
 7   DefensiveWinShares              24585 non-null  float64
 8   WinShares                       24585 non-null  float64
 9   FieldGoals                      24624 non-null  int64
 10  FieldGoalAttempts               24624 non-null  int64
 11  FieldGoalPercentage             24525 non-null  float64
 12  TwoPointFieldGoals              24624 non-null  int64
 13  TwoPointFieldGoalAttempts       24624 non-null  int64
 14  TwoPointFieldGoalPercentage     24496 non-null  float64
 15  EffectiveFieldGoalPercentage    24525 non-null  float64
 16  FreeThrows                      24624 non-null  int64
 17  FreeThrowAttempts               24624 non-null  int64
 18  FreeThrowPercentage             23766 non-null  float64
 19  Assists                         24624 non-null  int64
 20  PersonalFouls                   24624 non-null  int64
 21  Points                          24624 non-null  int64
dtypes: float64(10), int64(11), object(1)
memory usage: 4.1+ MB
```

```
df.isnull().sum()
#valores ausentes
```

	0
Ano	0
Posicao	0
Idade	8
Jogos	0
TrueShootingPercentage	86
FreeThrowRate	99
OffensiveWinShares	39
DefensiveWinShares	39
WinShares	39
FieldGoals	0
FieldGoalAttempts	0
FieldGoalPercentage	99
TwoPointFieldGoals	0
TwoPointFieldGoalAttempts	0
TwoPointFieldGoalPercentage	128
EffectiveFieldGoalPercentage	99
FreeThrows	0
FreeThrowAttempts	0
FreeThrowPercentage	858
Assists	0
PersonalFouls	0
Points	0

dtype: int64

Preparação dos Dados

Foi realizada uma análise dos valores ausentes presentes no dataset. Como a quantidade de registros com valores faltantes representa uma pequena parcela da base de dados, optou-se pela remoção dessas observações para simplificar o processo de preparação dos dados.

Além disso, a variável categórica "Posicao" foi convertida para formato numérico utilizando a técnica One-Hot Encoding, permitindo sua utilização pelos algoritmos de Machine Learning.

```
#removendo registros com valores ausentes

df = df.dropna()

print(df.shape)
```

```
(23697, 22)
```

```
#ajustando categórica "Posicao" para formato numérico

df = pd.get_dummies(df, columns=['Posicao'])

print(df.shape)
df.head()
```

```
(23697, 44)
```

	Ano	Idade	Jogos	TrueShootingPercentage	FreeThrowRate	OffensiveWinShares	DefensiveWinShares	WinShares	FieldGoals	Field
0	1950	31.0	63	0.368	0.467	-0.1	3.6	3.5	144	
1	1950	29.0	49	0.435	0.387	1.6	0.6	2.2	102	
2	1950	25.0	67	0.394	0.259	0.9	2.8	3.6	174	
3	1950	24.0	15	0.312	0.395	-0.5	-0.1	-0.6	22	
4	1950	24.0	13	0.308	0.378	-0.5	-0.1	-0.6	21	

5 rows × 44 columns

Definição das Variáveis

A variável WinShares foi definida como variável alvo, ou seja, o valor que o modelo buscará prever.

As demais variáveis do dataset foram utilizadas como atributos preditores.

```
#separação entre variáveis preditoras (X) e variável alvo (y)

X = df.drop('WinShares', axis=1)
y = df['WinShares']

print(X.shape)
print(y.shape)
```

```
(23697, 43)
(23697,)
```

Seleção de Atributos

O enunciado solicita a aplicação obrigatória de uma técnica de seleção ou extração de atributos.

Neste trabalho foi utilizada a técnica SelectKBest com a função estatística f_regression. Essa técnica seleciona os atributos que apresentam maior relação estatística com a variável alvo WinShares.

```
#importação da técnica de seleção de atributos
#utilizando a função estatística f_regression com 10 atributos

from sklearn.feature_selection import SelectKBest, f_regression

selector = SelectKBest(score_func=f_regression, k=10)

X_selected = selector.fit_transform(X, y)

print(X_selected.shape)
```

```
(23697, 10)
```

```
#exibição dos atributos escolhidos pela técnica SelectKBest
selected_columns = X.columns[selector.get_support()]

selected_columns
```

```
Index(['OffensiveWinShares', 'DefensiveWinShares', 'FieldGoals',
       'FieldGoalAttempts', 'TwoPointFieldGoals', 'TwoPointFieldGoalAttempts',
       'FreeThrows', 'FreeThrowAttempts', 'PersonalFouls', 'Points'],
      dtype='object')
```

Divisão entre Treino e Teste

Após a preparação dos dados, a base foi dividida em 75% para treinamento e 25% para teste, conforme solicitado no enunciado da atividade.

A base de treinamento foi utilizada para construir o modelo de Machine Learning e a base de teste foi utilizada para avaliar sua capacidade de previsão em dados não utilizados durante o treinamento.

```
#divisão da base de dados em treino e teste, 25% teste 75% treinamento
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_selected,
    y,
    test_size=0.25,
    random_state=42
)

print("Treino:", X_train.shape)
print("Teste:", X_test.shape)
```

```
Treino: (17772, 10)
Teste: (5925, 10)
```

Treinamento do Modelo

Após a preparação dos dados e a seleção dos atributos mais relevantes, foi utilizado o algoritmo Random Forest Regressor para realizar o treinamento do modelo.

Esse algoritmo foi escolhido por sua capacidade de lidar com relações não lineares entre variáveis e por apresentar bons resultados em problemas de regressão.

```
#importação do algoritmo Random Forest Regressor
from sklearn.ensemble import RandomForestRegressor

#criação do modelo
modelo = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
#treinamento do modelo utilizando a base de treino
modelo.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

Predição da Base de Teste

Após o treinamento, o modelo foi utilizado para realizar previsões sobre a base de teste, composta por 25% dos registros do dataset.

```
#realização das previsões utilizando a base de teste
predicoes = modelo.predict(X_test)

predicoes[:10]
```

```
array([0.771, 1.878, 1.296, 2.754, 0.4  , 0.056, 1.041, 7.206, 6.959,
       0.384])
```

Avaliação do Modelo

Para avaliar o desempenho do modelo foram utilizadas métricas de regressão.

As métricas escolhidas foram:

- MAE: erro absoluto médio
- RMSE: raiz do erro quadrático médio
- R^2: coeficiente de determinação

```
#importação das métricas de avaliação
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

#calculo das metricas do modelo
mae = mean_absolute_error(y_test, predicoes)

rmse = mean_squared_error(y_test, predicoes) ** 0.5

r2 = r2_score(y_test, predicoes)

print("MAE:", mae)
```

```
print("RMSE:", rmse)
print("R²:", r2)
```

```
MAE: 0.04959084227446256
RMSE: 0.11199032321243645
R²: 0.9987126955928989
```

Comparação entre Valores Reais e Previstos

A tabela abaixo apresenta uma comparação entre os valores reais de WinShares e os valores previstos pelo modelo para os primeiros registros da base de teste.

```
#comparação entre valores reais e valores previstos
resultado = pd.DataFrame({
    'Valor Real': y_test.values,
    'Valor Previsto': predicoes
})

resultado.head(20)
```

	Valor Real	Valor Previsto
0	0.8	0.771
1	1.9	1.878
2	1.3	1.296
3	2.9	2.754
4	0.4	0.400
5	0.0	0.056
6	1.1	1.041
7	7.2	7.206
8	7.0	6.959
9	0.4	0.384
10	8.8	8.641
11	3.4	3.295
12	0.2	0.300
13	-0.1	-0.102
14	0.4	0.370
15	1.5	1.547
16	1.0	1.095
17	7.9	7.933
18	8.6	8.589
19	6.5	6.472

Próximas etapas: Gerar código com `resultado` | New interactive sheet

Análise dos Resultados

Ao comparar os valores reais com os valores previstos pelo modelo, observa-se que as previsões ficaram muito próximas dos valores originais do dataset.

Por exemplo, para um jogador com WinShares real igual a 7,2, o modelo previu 7,206. Da mesma forma, para um valor real de 8,8, a previsão foi de 8,641. Essas diferenças são pequenas e indicam que o modelo conseguiu identificar adequadamente os padrões presentes nos dados.

Os resultados observados na amostra apresentada corroboram as métricas calculadas anteriormente, especialmente o alto valor de R^2 (0,9987) e os baixos valores de erro (MAE e RMSE), indicando que o modelo apresentou alta capacidade preditiva para os dados utilizados neste estudo.

Conclusão

Neste trabalho foi desenvolvido um modelo de Machine Learning para prever a variável WinShares utilizando o dataset NBA Players Stats.

Foram realizadas etapas de análise exploratória dos dados, tratamento de valores ausentes, transformação de variáveis categóricas, seleção de atributos e divisão entre conjuntos de treino e teste.

A técnica SelectKBest foi utilizada para selecionar os atributos mais relevantes para a variável alvo, enquanto o algoritmo Random Forest Regressor foi utilizado para realizar o treinamento do modelo.

Os resultados obtidos demonstraram excelente desempenho, apresentando baixos valores de erro e um coeficiente R^2 próximo de 1, indicando alta capacidade preditiva para a variável WinShares.

Dessa forma, conclui-se que técnicas de aprendizado supervisionado podem ser utilizadas com sucesso para estimar a contribuição de jogadores para as vitórias de suas equipes a partir de estatísticas históricas da NBA.